

TECH FORCE: GALACTIC
ARCHITECTURE

CORRUPÇÃO DE MEMÓRIA



9

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – UAF erro lógico.....	6
Código-fonte 2 – UAF má gestão de memória compartilhada.....	7
Código-fonte 3 – UFA suposição incorreta.....	7
Código-fonte 4 – Liberação dupla falha lógica	8
Código-fonte 5 – Liberação dupla gestão inadequada de ponteiros	9
Código-fonte 6 – Condição de corrida.....	10
Código-fonte 7 – Ponteiros pendentes	12

EMANSP

SUMÁRIO

CORRUPÇÃO DE MEMÓRIA.....	4
1 INTRODUÇÃO	4
2 TIPOS COMUNS DE CORRUPÇÃO DE MEMÓRIA	4
3 BUFFER OVERFLOW	5
4 USE-AFTER-FREE (UAF).....	5
5 <i>DOUBLE FREE</i>	8
6 CONDIÇÃO DE CORRIDA	9
7 PONTEIROS PENDENTES	11
CONCLUSÃO	13
REFERÊNCIAS.....	14

CORRUPÇÃO DE MEMÓRIA

1 INTRODUÇÃO

Em nosso mundo cada vez mais digitalizado, no qual sistemas computacionais tornaram-se onipresentes e essenciais em quase todas as facetas de nossas vidas, a integridade e segurança desses sistemas é de suma importância. No coração desses sistemas, reside a memória, um componente fundamental que armazena e recupera informações.

No entanto, assim como qualquer componente crítico, a memória não está imune a falhas e vulnerabilidades. A corrupção de memória, um termo que muitos podem ter ouvido, mas poucos realmente compreendem, refere-se a situações em que a memória de um sistema é alterada sem intenção, resultando em consequências inesperadas, muitas vezes prejudiciais. Esta corrupção pode surgir de uma variedade de fontes, desde erros de programação inadvertidos até ataques deliberados por agentes maliciosos.

Este capítulo busca desvendar o mistério em torno da corrupção de memória, explorando seus tipos, causas, efeitos e, mais importante, como podemos preveni-la. Ao entender este fenômeno, estaremos melhor equipados para construir sistemas mais robustos, seguros e confiáveis, garantindo que a revolução digital continue a prosperar de maneira segura e eficiente.

2 TIPOS COMUNS DE CORRUPÇÃO DE MEMÓRIA

A corrupção de memória, um perturbador obstáculo no universo da programação, manifesta-se de diversas maneiras, e compreender suas variações é essencial para diagnosticar e mitigar seus efeitos. Esta seção examinará cinco tipos comuns de corrupção de memória, que são frequentemente encontrados em várias aplicações e sistemas.

Desde o ardiloso *buffer overflow*, que ocorre quando os dados excedem os limites de uma área de memória alocada, até as nuances do "uso após liberação" e "liberação dupla", este segmento busca lançar luz sobre essas armadilhas técnicas. Adicionalmente, abordaremos as "condições de corrida", onde operações concorrentes

resultam em comportamentos indesejados, e os "ponteiros pendentes", um clássico erro em que as referências a locais de memória já liberados persistem. Ao adentrar este segmento, o leitor se equipará com conhecimento crucial para entender, prevenir e combater esses comuns, mas potencialmente devastadores, problemas de memória.

3 BUFFER OVERFLOW

Não vamos nos aprofundar em *buffer overflow* neste capítulo, porém cabe já dar uma breve descrição sobre esta falha.

Um "*buffer overflow*" (ou "transbordamento de dados", em português) refere-se a um incidente onde um programa, ao escrever dados em um *buffer*, ultrapassa os limites do mesmo e começa a escrever em uma área adjacente da memória. *Buffers* são áreas da memória reservadas para armazenar temporariamente dados, e são de tamanho fixo. Se o programa tentar armazenar mais dados no *buffer* do que ele pode conter, os dados adicionais "transbordam" para os endereços de memória adjacentes.

Este tipo de vulnerabilidade pode ter várias consequências:

1. Corrupção de dados: Dados adjacentes na memória podem ser corrompidos ou sobrescritos.
2. Falhas do sistema: Pode levar a comportamentos imprevisíveis do programa, resultando em falhas ou travamentos.
3. Execução de código malicioso: Malfeitores podem explorar *buffer overflow* para inserir e executar código malicioso. Ao cuidadosamente criar entradas que causam um *overflow*, um atacante pode sobrescrever partes críticas da memória (como a pilha de retorno) para redirecionar a execução do programa para um código de sua escolha.

Os *buffer overflows* foram, durante muitos anos, uma das vulnerabilidades de segurança mais exploradas em softwares, levando à adoção generalizada de diversas medidas defensivas em sistemas operacionais e compiladores.

4 USE-AFTER-FREE (UAF)

“*Use-After-Free*” (ou “uso após liberação”, em português) é uma vulnerabilidade que ocorre quando um programa tenta acessar um recurso de memória após ter sido liberado (ou “desalocado”). Este erro permite que um adversário possa fazer uso do espaço de memória livre e armazenar outros dados, ou até mesmo marcar essa área da memória como inválida.

A falha catalogada como CWE-416 possui consequências severas como a corrupção de dados, falhas do sistema e execução de código malicioso. Para compreender UAF, é importante ter um entendimento geral sobre gerenciamento de memória, particularmente como C e C++. Nestas linguagens, os desenvolvedores frequentemente usam funções como `malloc()` para alocar e `free()` para liberar um determinado espaço de memória. Quando um programa continua a usar uma localização da memória após ela ter sido liberada ou desalocada, a falha UAF pode ocorrer e ser explorada.

Alguns erros comuns que podem ocorrer em um programa e permite que esse tipo de falha ocorra são erros lógicos, má gestão da memória compartilhada e suposições incorretas. O primeiro são simplesmente falhas na lógica do programa. Estes erros acontecem quando um programador, por engano, acredita que uma área de memória ainda é válida e tenta acessá-la após liberá-la:

```
int* ptr = (int*)malloc(sizeof(int));  
*ptr = 10;  
free(ptr);  
*ptr = 20; // Uso após liberação
```

Código-fonte 1 – UAF erro lógico
Fonte: Elaborado pelo autor (2023)

Neste exemplo, a memória é alocada e, em seguida, liberada. No entanto, o programador tenta acessar essa memória novamente, levando a um UAF.

Por sua vez o segundo caso, má gestão de memória compartilhada, ocorre quando várias partes de um programa compartilham uma referência à mesma memória, uma parte pode liberá-la enquanto outra ainda espera usá-la.

```
int* ptr1 = (int*)malloc(sizeof(int));
int* ptr2 = ptr1;
free(ptr1);
*ptr2 = 30; // Uso após liberação, pois ptr2 e ptr1 apontavam para o mesmo
local de memória
```

Código-fonte 2 – UAF má gestão de memória compartilhada
Fonte: Elaborado pelo autor (2023)

Neste exemplo, ptr1 e ptr2 apontam para o mesmo local de memória. A memória é liberada através de ptr1, mas ptr2 ainda tenta acessá-la.

Já a terceira causa, suposições incorretas, acontece quando um programador supõe que uma função ou método não libera certa memória, quando, na verdade, ela faz.

```
void liberaMemoria(int** ptr) {
    free(*ptr);
    *ptr = NULL;
}

int main() {
    int* meuPtr = (int*)malloc(sizeof(int));
    *meuPtr = 40;
    liberaMemoria(&meuPtr);
    *meuPtr = 50; // Uso após liberação
    return 0;
}
```

Código-fonte 3 – UFA suposição incorreta
Fonte: Elaborado pelo autor (2023)

Neste exemplo, a função “liberaMemoria” libera a memória apontada por meuPtr e também define o ponteiro como NULL. No entanto, na função “main”, o programador, não estando ciente ou esquecendo-se da liberação da memória dentro de liberaMemoria, tenta usar meuPtr novamente.

Esses erros, embora pareçam simples exemplos didáticos, podem ser muito sutis e difíceis de detectar em programas complexos, onde o fluxo de memória é complicado e há muitas interações entre diferentes partes do código.

5 DOUBLE FREE

A vulnerabilidade de Double Free (liberação dupla) ocorre quando um programa tenta liberar (ou desalocar) um mesmo endereço de memória mais de uma vez. Em linguagens de programação como C e C++, onde os desenvolvedores têm controle direto sobre a alocação e desalocação de memória, essa vulnerabilidade pode surgir devido a falhas na gestão de memória.

As duas causas principais para esse tipo de falha ocorrer são erros lógicos e gestão inadequada de ponteiros.

No primeiro caso, falhas de programação podem levar a situações em que a memória é liberada mais de uma vez. Um exemplo simples disso encontra-se abaixo:

```
int* ptr = (int*) malloc(sizeof(int));
*ptr = 100;
free(ptr);
free(ptr); // Liberação dupla
```

Código-fonte 4 – Liberação dupla falha lógica
Fonte: Elaborado pelo autor (2023)

Neste código simples, a memória é alocada e, depois, liberada duas vezes consecutivas, causando uma liberação dupla.

A vulnerabilidade de liberação dupla pode ter várias consequências prejudiciais, como a corrupção de memória, na qual o gerenciador de memória mantém metainformações sobre blocos de memória alocados no *heap*. Uma liberação dupla pode corromper essa metainformação, levando a um comportamento imprevisível do programa.

- Falhas no programa: Liberações duplas frequentemente causam falhas no programa, como falha de segmentação.
- Execução arbitrária de código: Em certas situações, atacantes podem explorar uma vulnerabilidade de liberação dupla para executar código arbitrário. Ao controlar os dados inseridos na memória entre as duas liberações, um atacante pode manipular estruturas de dados do gerenciador de memória de modo a ganhar controle sobre a execução do programa.

A segunda causa mais comum é a gestão inadequada de ponteiros. Os ponteiros são variáveis que armazenam o endereço de outra variável em memória. Na programação de baixo nível, especialmente em linguagens como C e C++, ponteiros são frequentemente usados para acessar e manipular a memória diretamente. Uma gestão inadequada de ponteiros pode ocorrer quando os programadores não controlam ou rastreiam de maneira eficaz os endereços de memória que esses ponteiros referenciam, levando a uma série de problemas, incluindo a vulnerabilidade de liberação dupla.

```
int* ptr1 = (int*)malloc(sizeof(int));
int* ptr2 = ptr1;
*ptr1 = 42;
// Em algum ponto no programa, a memória é liberada usando ptr1.
free(ptr1);
ptr1 = NULL; // Correto: Definimos ptr1 como NULL após liberar.
// Mais adiante no programa, esquecemos que ptr2 aponta para a mesma
memória que ptr1 apontava.
free(ptr2); // Liberação dupla!
```

Código-fonte 5 – Liberação dupla gestão inadequada de ponteiros
Fonte: Elaborado pelo autor (2023)

Neste exemplo, ptr1 e ptr2 apontam para o mesmo bloco de memória alocado. A memória é liberada usando ptr1, e depois, inadvertidamente, é liberada novamente usando ptr2. Se ptr2 tivesse sido definido como NULL após a liberação inicial, o segundo free() não teria causado uma liberação dupla.

6 CONDIÇÃO DE CORRIDA

Uma condição de corrida ocorre quando duas ou mais operações concorrentes acessam um recurso compartilhado de maneira que a sequência final de acessos afeta o resultado, e a ordem desses acessos é determinada pela maneira como as operações são agendadas. No contexto de corrupção de memória, uma condição de corrida pode levar a resultados imprevisíveis, como leitura de dados não inicializados, sobreposição de memória ou até mesmo falhas no sistema.

Condições de corrida surgem, principalmente, em ambientes *multithreaded* (ou multiprocessados), onde várias *threads* ou processos acessam e/ou modificam a mesma região de memória concorrentemente.

Imagine uma aplicação que tem uma variável compartilhada chamada saldo, que representa o saldo de uma conta bancária. Há duas threads: uma que adiciona dinheiro à conta (depósito) e outra que retira dinheiro (saque).

Sem medidas apropriadas para prevenir condições de corrida, o seguinte cenário pode ocorrer:

1. A *thread* de depósito lê o valor de saldo (digamos, \$100).
2. Antes que a *thread* de depósito possa atualizar o valor de saldo, a thread de saque lê o mesmo valor (\$100).
3. A *thread* de depósito adiciona \$50, atualizando o saldo para \$150.
4. A *thread* de saque, pensando que o saldo é ainda \$100, retira \$50, atualizando o saldo para \$50 em vez de \$100 (\$150 - \$50).

No final deste cenário, a conta tem \$50 quando deveria ter \$100, levando a uma corrupção de dados.

Vamos agora para o código. Imagine um cenário no qual duas *threads* estão operando sobre um objeto dinamicamente alocado na memória:

```
int contador = 0;
void incrementar() {
    for (int i = 0; i < 1000000; i++) {
        contador++;
    }
}
```

Código-fonte 6 – Condição de corrida
Fonte: Elaborado pelo autor (2023)

Se duas threads executam a função incrementar() simultaneamente, esperaríamos que, ao final da execução, o valor de contador fosse 2000000. No entanto, devido à condição de corrida, o valor pode ser menor que o esperado.

1. A *thread* A lê o valor de contador (digamos, 0).

2. Antes que a *thread* A possa incrementar e armazenar o novo valor, a *thread* B lê o mesmo valor de contador (ainda 0).
3. A *thread* A incrementa o valor lido e o armazena (agora é 1).
4. A *thread* B, que leu 0, também incrementa esse valor e o armazena. O valor ainda é 1 em vez de 2.

Essa discrepância de 1 pode não parecer muito em uma única iteração, porém, considerando muitas iterações e múltiplas *threads*, o impacto acumulado pode ser significativo.

7 PONTEIROS PENDENTES

Dentro do domínio de corrupção de memória em segurança da informação, uma das vulnerabilidades notórias é a de "ponteiros pendentes" (ou "dangling pointers", em inglês). Essa vulnerabilidade ocorre quando um ponteiro é usado após o recurso ao qual ele apontava ter sido desalocado, tornando o ponteiro "pendente". O uso inseguro de ponteiros pendentes pode resultar em comportamentos indefinidos, incluindo vazamento de informação, corrupção de dados e até execução arbitrária de código.

Ponteiros pendentes são geralmente introduzidos devido a falhas na lógica do programa ou omissões onde o programador não atualiza ou zera ponteiros após a desalocação do recurso. Quando um recurso é desalocado, o espaço de memória que ele ocupava pode ser reutilizado para outros propósitos. Se um ponteiro pendente for usado posteriormente, ele pode apontar para essa nova alocação de memória, o que pode ter implicações de segurança.

Vamos considerar o seguinte exemplo escrito em C:

```
#include <stdlib.h>

int main() {
    int *ptr = malloc(10 * sizeof(int));

    if (ptr == NULL) {
        exit(1);
    }

    free(ptr);

    *ptr = 123;

    return 0;
}
```

Código-fonte 7 – Ponteiros pendentes
Fonte: Elaborado pelo autor (2023)

Neste código, a linha `int *ptr = malloc(10 * sizeof(int));` aloca espaço de memória suficiente para 10 inteiros e retorna um ponteiro para o início desse bloco de memória.

- `sizeof(int)` retorna o tamanho (em bytes) de um tipo inteiro no sistema em que o código é compilado. Multiplicar por 10 significa que estamos solicitando memória para armazenar 10 inteiros. E `ptr` é agora um ponteiro que aponta para o início deste bloco de memória.
- `if (ptr == NULL) {...}` é uma verificação para garantir que a alocação de memória foi bem-sucedida. Se `malloc` falhar (por exemplo, devido à falta de memória disponível), ele retornará `NULL`. Se isso acontecer, o programa sai imediatamente com um código de erro 1.
- `free(ptr)`, esta linha desaloca, ou libera, a memória que foi anteriormente alocada com `malloc`. Após esta chamada, o bloco de memória que `ptr` apontava é devolvido ao sistema, e essa região de memória pode agora ser reutilizada por chamadas subsequentes de `malloc` ou para outros propósitos.

No entanto, é importante notar que, embora a memória tenha sido liberada, o valor de `ptr` (o endereço que ele detém) não foi alterado. Portanto, `ptr` torna-se um "ponteiro pendente".

`*ptr = 123;` Por fim, esta linha tenta escrever o valor 123 na memória apontada por `ptr`. No entanto, como vimos, `ptr` é um ponteiro pendente, pois a memória que ele apontava foi liberada. O resultado dessa operação é indefinido: pode resultar em um comportamento não intencional, um crash do programa ou, em um contexto de

vulnerabilidade de segurança, permitir que um atacante corrompa a memória de maneira benéfica para seus objetivos maliciosos.

CONCLUSÃO

A corrupção de memória, um dos vetores de ataque mais antigos e persistentes na segurança de sistemas computacionais, permanece sendo uma preocupação preeminente em ambientes modernos. As vulnerabilidades relacionadas à corrupção de memória, como demonstrado através de nossas explorações, ocorrem quando um programa escreve dados fora dos limites apropriados de um bloco de memória alocado, permitindo a manipulação maliciosa da memória do programa para obter comportamentos indesejados.

Durante nossa discussão, exploramos falhas como overflow de buffer, que permite aos atacantes sobrescrever áreas adjacentes da memória; uso após livre, onde um programa tenta acessar um bloco de memória depois de já ter sido liberado; liberação dupla, que ocorre quando um bloco de memória é liberado mais de uma vez; ponteiro pendente, uma situação em que um ponteiro ainda aponta para uma área de memória após sua liberação; e condição de corrida, que se manifesta quando a ordem de execução de threads ou processos é mal gerenciada.

Além de entender essas vulnerabilidades, é crucial reconhecer as técnicas de exploração e os métodos práticos de prevenção. A abordagem proativa na programação, o uso de ferramentas de análise, e a aplicação de práticas de codificação segura são medidas preventivas essenciais. Em conclusão, a corrupção de memória, apesar da evolução das técnicas de defesa, continua a ser uma ameaça significativa, exigindo vigilância e educação contínua para desenvolvedores e profissionais de segurança.

REFERÊNCIAS

ERICKSON, J. **Hacking**: The Art of Exploitation. 2. ed. San Francisco: No Starch Press, 2008.

SEACORD, R. C. **Secure Coding in C and C++**. Boston: Addison-Wesley, 2005.

ZALEWSKI, M. **The Tangled Web**: A Guide to Securing Modern Web Applications. San Francisco: No Starch Press, 2011.

EXEMPLO